



Dispotree Development

Week 1 Progress Report | December 13-19, 2025

This week we built the complete Dispotree backend system from the ground up, establishing a production-ready foundation with an extensible architecture designed for rapid feature development.

Technical Metrics

706

Tests Passing

55+

API Endpoints

15

Database Models

4

Plugin Types

20

AI Agent Tools

17K+

Lines of Code

Complete System Overview - How Everything Works

Dispotree is a real estate deal management platform that automates the entire lifecycle of property deals from ingestion through offer acceptance. The system is built on a modular plugin architecture that allows each component to work independently while communicating through a central event system.

The Big Picture

At its core, Dispotree solves a specific problem: **How do you efficiently match incoming property deals with hedge fund investors who have specific buying criteria?**

The answer involves several interconnected systems:

1. **Deal Ingestion** - Getting property data into the system from multiple sources
2. **Intelligent Scoring** - Automatically evaluating each property against fund requirements
3. **Automation Engine** - Taking action based on scores and events
4. **Marketplace** - Presenting matched deals to the right buyers
5. **Transaction Management** - Handling offers and contract signing

1. Deal Ingestion System - How Properties Enter the Platform

The first challenge is getting property data into the system. Different sources provide data in different formats, so we built a plugin-based ingestion system that normalizes everything into a standard format.

How Each Source Works:

CSV Upload Plugin

What it does: Accepts bulk property uploads via CSV files.

How it works:

1. User uploads a CSV file through the `POST /api/plugins/sources/csv` endpoint
2. System validates the file format and required columns (address, city, state, zip, asking price)
3. Each row is parsed and mapped to the standard Property schema
4. Properties are created in the database with `source: 'csv'` tag
5. For each property, a `deal.created` event is emitted to trigger automations

Use case: Wholesalers who have spreadsheets of properties from their marketing campaigns.

API Push Plugin

What it does: Accepts real-time property submissions via REST API.

How it works:

1. External system calls `POST /api/plugins/sources/api` with property JSON
2. Request is validated against the Property schema using Joi validation
3. Property is created with `source: 'api'` tag
4. `deal.created` event fires, triggering scoring and automations

Use case: Integration with CRMs, lead generation tools, or partner systems.

Email Parser Plugin

What it does: Monitors an email inbox and extracts property data from incoming emails.

How it works:

1. Connects to configured IMAP server (Gmail, Outlook, etc.)
2. Polls for new emails matching configured criteria (sender, subject patterns)
3. Parses email body and attachments using AI to extract property details
4. Creates property records with `source: 'email'`
5. Marks email as processed to avoid duplicates

Use case: Receiving deals from bird dogs or scouts who send leads via email.

Webhook Receiver Plugin

What it does: Accepts incoming webhooks from external platforms.

How it works:

1. Exposes endpoint `POST /api/plugins/sources/webhook/:sourceId`
2. Validates webhook signature (if configured) for security
3. Maps incoming payload to Property schema using configurable field mappings
4. Creates property with `source: 'webhook'`

Use case: Zapier integrations, PropStream exports, or custom integrations.

Firecrawl Web Scraper Plugin

What it does: Scrapes property listings from auction sites and listing platforms.

How it works:

1. Uses Firecrawl API to navigate to configured URLs
2. Extracts property data using CSS selectors or AI extraction
3. Handles pagination to scrape multiple listings
4. Deduplicates against existing properties by address
5. Creates new properties with `source: 'firecrawl'`

Use case: Monitoring auction.com, Hubzu, or county tax sale sites.

2. Property Enrichment Pipeline - Automatic Data Enhancement

When a deal enters the system from any source, it goes through an automatic enrichment pipeline that enhances the raw property data with market intelligence, valuations, and scoring.

Enrichment Flow

Raw Property Data



Enrichment Pipeline



Enriched Deal

What Gets Enriched Automatically:

1. Address Standardization & Validation

What happens:

- ✓ Address parsed into components (street, city, state, zip)
- ✓ USPS standardization applied for consistent formatting
- ✓ Geocoding to get latitude/longitude coordinates
- ✓ County and school district identification
- ✓ Duplicate detection against existing properties

2. Property Characteristics Lookup

What happens:

- ✓ Bedrooms, bathrooms, square footage fetched from public records
- ✓ Year built and lot size retrieved
- ✓ Property type confirmed (SFR, Duplex, Condo, etc.)
- ✓ Construction type and foundation details
- ✓ Garage, pool, and other amenities identified

3. Market Data & Valuation

What happens:

- ✓ Current market value estimate (Zestimate-style)
- ✓ Tax assessed value from county records
- ✓ Last sale price and date
- ✓ Price per square foot for the area
- ✓ Days on market for comparable listings
- ✓ Neighborhood price trends (appreciation/depreciation)

4. ARV & Equity Calculations

What happens:

- ✓ After Repair Value (ARV) estimated from comps
- ✓ Equity percentage calculated: $(ARV - \text{Asking Price}) / ARV$
- ✓ Potential profit margin estimated
- ✓ Repair cost estimates based on property condition

- ✓ Maximum Allowable Offer (MAO) calculated using 70% rule

5. Comparable Sales Analysis

What happens:

- ✓ 3-5 comparable properties identified within 1 mile
- ✓ Recent sales (last 6 months) prioritized
- ✓ Adjustments made for bedrooms, bathrooms, square footage
- ✓ Comp confidence score calculated
- ✓ Comp data stored for due diligence review

6. Ownership & Title Information

What happens:

- ✓ Current owner name and mailing address
- ✓ Ownership duration (how long they've owned)
- ✓ Mortgage information if available
- ✓ Lien and encumbrance flags
- ✓ Foreclosure or pre-foreclosure status

7. ML Quality Scoring

What happens:

- ✓ TensorFlow.js model analyzes all property features
- ✓ Predicts likelihood of deal acceptance (0-100%)
- ✓ Compares to historical successful deals
- ✓ Assigns deal quality tier (A, B, C, D)
- ✓ Prediction logged for model improvement

8. Buy Box Matching & Scoring

What happens:

- ✓ Property scored against ALL active fund buy boxes
- ✓ Weighted criteria matching (location, price, equity, etc.)
- ✓ Top-scoring funds identified
- ✓ Match explanations generated (why it matched/didn't)
- ✓ Auto-assignment to matching funds triggered

Data Type	What's Included
Zestimate	Property valuation, range (low/high), rent Zestimate
Skip Trace	Owner name, phone numbers, emails, mailing address, occupancy
Property Details	Beds, baths, sqft, lot size, year built, property type
Price History	All listing/sale events, dates, prices, agents
Tax History	Annual tax assessments, amounts paid
Tax Charts	Visual assessment & payment trends over time
Comparables	Similar nearby properties with prices, details, distance
Rental Trends	Market-level median rent, listings, trends

Enrichment Speed: Most enrichment completes within 2-5 seconds of deal ingestion. Complex lookups (comps, owner info) may take up to 30 seconds but run asynchronously without blocking the pipeline.

3. Buy Box Scoring Engine

Once a property enters the system, it needs to be evaluated against each hedge fund's investment criteria. This is where the Buy Box Scoring Engine comes in.

What is a Buy Box?

A "Buy Box" defines a hedge fund's ideal investment property. It includes criteria like:

- ✓ Geographic focus (specific states, cities, or zip codes)
- ✓ Property type (Single Family, Duplex, Multifamily)
- ✓ Price range (min/max asking price)
- ✓ Equity requirements (minimum equity percentage)
- ✓ Property condition (year built, square footage)
- ✓ ARV thresholds (After Repair Value requirements)

How Scoring Works:

Step 1: Criteria Definition

Each fund defines their buy box with weighted criteria. For example:

- ✓ State must be TX, FL, AZ, or GA (weight: 20 points)
- ✓ Property type must be Single Family (weight: 15 points)
- ✓ Asking price between \$100K-\$350K (weight: 25 points)

- ✓ Equity percentage $\geq 20\%$ (weight: 25 points)
- ✓ Year built ≥ 1980 (weight: 15 points)

Step 2: Automatic Scoring

When a property is created, the system automatically:

1. Loads all active buy boxes from the database
2. Evaluates the property against each buy box's criteria
3. For each matching criterion, adds its weight to the score
4. Calculates final score as percentage (0-100)
5. Stores all scores in the property record

Step 3: Score Storage and Events

After scoring completes:

- ✓ All fund scores are stored in the `buyBoxScores` JSON field
- ✓ The highest score is stored in `topScore` for quick filtering
- ✓ The best-matching fund ID is stored in `topFundId`
- ✓ A `score.calculated` event fires with all score data

Example: A property in Texas, Single Family, asking \$250K, with 25% equity, built in 1995 would score:

- State match: +20 points
- Property type match: +15 points
- Price in range: +25 points
- Equity $\geq 20\%$: +25 points
- Year built ≥ 1980 : +15 points
- **Total: 100/100 (perfect match)**

4. Automation Engine - Event-Driven Actions

The automation engine listens for events and executes configurable actions when conditions are met. This eliminates manual work and ensures consistent processing.

How the Event System Works:

Event Emission

Events are emitted at key points in the system:

Event	When It Fires	Data Included
<code>deal.created</code>	New property saved to database	Full property object, source type, timestamp
<code>deal.updated</code>	Property data modified	Property object, list of changed fields
<code>score.calculated</code>	Buy box scoring completes	Property, all scores, top score, top fund
<code>offer.submitted</code>	User submits an offer	Offer details, user info, property
<code>offer.accepted</code>	Offer marked as accepted	Offer, property, acceptance details
<code>document.uploaded</code>	File attached to property	Document metadata, property ID

Automation Structure

Each automation has three parts:

1. **Trigger:** Which event to listen for
2. **Conditions:** Rules that must be true for actions to execute
3. **Actions:** What to do when conditions are met

Condition Operators

Conditions support these operators:

Operator	Example	Description
<code>equals</code>	state equals "TX"	Exact match
<code>not_equals</code>	status not_equals "sold"	Does not match
<code>>=</code> , <code><=</code> , <code>></code> , <code><</code>	topScore >= 80	Numeric comparison
<code>in</code>	propertyType in ["SFR", "Duplex"]	Value in list
<code>between</code>	askingPrice between [100000, 350000]	Range check
<code>contains</code>	address contains "Main St"	String contains

Available Actions

When conditions are met, these actions can execute:

Action	What It Does	Configuration
<code>notify</code>	Sends alert via SMS, email, or webhook	Channel, message template with {{variables}}
<code>assign</code>	Assigns deal to specific fund or user	Target fund ID or user ID

<code>tag</code>	Adds tags to the property	Array of tag strings
<code>updateField</code>	Changes a property field value	Field name, new value
<code>startWorkflow</code>	Initiates a multi-step workflow	Workflow ID to start
<code>webhook</code>	Calls external URL	URL, HTTP method, payload template
<code>createTask</code>	Creates follow-up task	Description, assignee, due date

Real Example: "When a deal scores 85+ for Fund ABC, automatically tag it as 'hot-lead', assign it to that fund, and send an SMS/email notification to the acquisitions team."

5. Multi-Step Workflow System

While automations handle simple trigger-action patterns, workflows orchestrate complex multi-step processes that may span days or weeks.

How Workflows Differ from Automations:

Feature	Automation	Workflow
Steps	Single action or parallel actions	Multiple sequential/parallel steps
State	Stateless (fire and forget)	Stateful (persisted to database)
Duration	Milliseconds	Hours, days, or weeks
Waiting	No waiting capability	Can wait for events or timeouts
Branching	No conditional branching	If/else logic between steps

Workflow Execution Process:

Step 1: Workflow Initiation

A workflow starts when:

- ✓ An automation's `startWorkflow` action fires
- ✓ A user manually triggers it via API
- ✓ Another workflow step triggers it

The system creates a `WorkflowExecution` record with status "running" and current step = 1.

Step 2: Step Execution

For each step, the workflow engine:

1. Loads the step configuration (action type, parameters)
2. Executes the step's action (notify, create task, call API, etc.)
3. Checks if step has a wait condition
4. If no wait, immediately moves to next step
5. If wait required, pauses and stores state to database

Step 3: Wait Conditions

Workflows can pause and wait for:

- ✓ **Event:** Wait until specific event fires (e.g., document.uploaded)
- ✓ **Timeout:** Wait for specified duration (e.g., 48 hours)
- ✓ **Approval:** Wait for user to approve/reject
- ✓ **External:** Wait for webhook callback

While waiting, the workflow state is persisted to PostgreSQL, so it survives server restarts.

Step 4: Conditional Branching

Steps can specify different next steps based on conditions:

- ✓ If ROI > 15%, go to "Generate Offer" step
- ✓ Else, go to "Reject Deal" step

This allows complex decision trees within a single workflow.

Step 5: Completion

When the final step completes:

- ✓ Workflow status set to "completed"
- ✓ Completion timestamp recorded
- ✓ `workflow.completed` event emitted
- ✓ Audit log finalized with all step results

6. ML Prediction System - AI-Powered Deal Scoring

Beyond rule-based buy box scoring, the ML system learns from historical data to predict deal quality and fund preferences.

How the ML Pipeline Works:

Training Data Collection

The system collects training data from:

- ✓ **Fund Feedback:** When funds accept/reject deals, feedback is recorded
- ✓ **Offer Outcomes:** Which offers get accepted vs rejected
- ✓ **User Behavior:** Which deals get views, likes, and offers
- ✓ **Deal Velocity:** How quickly deals move through the pipeline

Each data point is stored in the `FundFeedback` model with the property features and outcome.

Model Training

Using TensorFlow.js, the system trains neural network models:

1. Collects all feedback data from database
2. Extracts features: price, equity, location, property type, etc.
3. Normalizes features to 0-1 scale
4. Trains model to predict: Will this deal be accepted?
5. Saves model weights to `ModelVersion` table
6. Records training metrics in `TrainingRun`

Prediction Generation

When new deals enter the system:

1. Property features are extracted
2. Features normalized using same scalers as training
3. Model generates prediction score (0-1 probability)
4. Score stored in `MLPrediction` table
5. Prediction linked to property for analytics

Model Versioning and A/B Testing

The system supports multiple model versions:

- ✓ Each model version tracked with accuracy metrics
- ✓ Can run A/B tests between model versions
- ✓ Automatic rollback if new model underperforms
- ✓ Predictions logged with model version for analysis

7. LangChain AI Agent - Conversational Interface

The AI Agent provides a natural language interface to the entire system, allowing users to query data and perform actions through conversation.

Agent Architecture:

Tool System

The agent has access to 20 tools that map to system capabilities:

Tool Category	Examples	What It Does
Property Search	<code>searchProperties</code> , <code>getPropertyDetails</code>	Find and retrieve property information
Analytics	<code>getDashboardStats</code> , <code>getConversionRates</code>	Pull analytics and metrics
Buy Box	<code>listBuyBoxes</code> , <code>scoreProperty</code>	Manage and evaluate buy boxes
Offers	<code>createOffer</code> , <code>listOffers</code>	Manage deal offers
Documents	<code>searchDocuments</code> , <code>analyzeContract</code>	Work with attached documents

Conversation Memory

The agent maintains conversation context:

- ✓ Stores conversation history in `ConversationHistory` model
- ✓ Remembers previous questions and answers
- ✓ Can reference earlier context ("What about that property in Texas?")
- ✓ Supports multi-turn conversations for complex queries

Example Interactions

User Says	Agent Does
"Show me deals in Texas under \$300K"	Calls <code>searchProperties</code> with state=TX, maxPrice=300000
"Which funds would be interested in this deal?"	Calls <code>scoreProperty</code> against all buy boxes, returns top matches
"What's our conversion rate this month?"	Calls <code>getConversionRates</code> with date range, formats response
"Draft an offer for \$180K on property #123"	Calls <code>createOffer</code> with amount and property ID

8. Contract Signing System

The contract system handles the complete document lifecycle from template creation through multi-party signing.

How Contract Signing Works:

Template Management

Contracts start as reusable templates:

- ✓ Templates stored with placeholder variables like `{{buyerName}}` , `{{propertyAddress}}`
- ✓ Support for multiple template versions
- ✓ Templates organized by type: Purchase Agreement, Assignment, etc.
- ✓ Admin interface to create and edit templates

Contract Generation

When a contract is needed:

1. System loads the appropriate template
2. Pulls data from Property, User, and Offer records
3. Replaces all placeholders with actual values
4. Generates PDF preview
5. Creates Contract record with status "pending_signatures"

Signature Collection

Multi-party signing process:

1. System identifies required signers (buyer, seller, agent)
2. Sends signing requests via email with secure links
3. Each signer accesses contract through unique token
4. Captures signature (typed name or drawn signature)
5. Records timestamp, IP address, and user agent
6. Updates contract status as signatures are collected

Completion and Storage

When all parties have signed:

- ✓ Final PDF generated with all signatures embedded
- ✓ Contract status updated to "executed"
- ✓ All parties receive copy via email
- ✓ Document linked to property record

- ✓ Complete audit trail available for compliance

9. Marketplace System

The marketplace connects deal providers with fund buyers through a structured interface.

User Types and Permissions:

Role	Can Do	Cannot Do
Admin	Everything: manage users, funds, settings	N/A
Fund User	View matched deals, submit offers, manage buy box	Modify other funds' data
Regular User	Browse deals, submit basic offers	Access fund-specific features

Deal Distribution:

How Deals Reach Funds

1. Property enters system and gets scored against all buy boxes
2. System identifies which funds have high match scores
3. Deals appear in each fund's "Matched Deals" queue
4. Funds see deals sorted by match score (best matches first)
5. Fund users can like, pass, or submit offers
6. Actions tracked in `DealAction` for analytics

10. Analytics System

The analytics system provides insights into deal flow, conversion rates, and performance metrics.

Available Metrics:

Dashboard Overview (`/api/analytics/overview`)

- ✓ Total properties in system
- ✓ Active listings (not sold/withdrawn)
- ✓ Pending offers count
- ✓ Deals created today/this week/this month
- ✓ Total offer value
- ✓ Average days on market

Source Performance (`/api/analytics/sources`)

Breaks down metrics by deal source:

- ✓ Deals per source (CSV, API, Email, Webhook, Firecrawl)
- ✓ Conversion rate by source (deals that got offers)
- ✓ Average deal quality score by source
- ✓ Time from ingestion to first offer by source

Conversion Funnel (`/api/analytics/conversions`)

Tracks the deal journey:

- ✓ View → Like rate
- ✓ Like → Offer rate
- ✓ Offer → Accepted rate
- ✓ Average time at each funnel stage
- ✓ Drop-off analysis

Deal Analytics (`/api/analytics/deals`)

Property-level insights:

- ✓ Deals by state, property type, price range
- ✓ Average ARV, asking price, equity percentage
- ✓ Top performing markets
- ✓ Trend analysis over time

11. Security and Authentication

JWT Authentication Flow:

Login Process

1. User submits email + password to `POST /api/auth/login`
2. System verifies password against bcrypt hash (12 rounds)
3. If valid, generates access token (15 min expiry) and refresh token (7 day expiry)
4. Tokens returned to client for storage

Request Authentication

1. Client includes access token in Authorization header
2. `authenticate` middleware validates token signature
3. Checks token expiration
4. Attaches user object to request for downstream use

Token Refresh

- 1. When access token expires, client calls `POST /api/auth/refresh`
- 2. Includes refresh token in request body
- 3. System validates refresh token and issues new access token
- 4. No password required for refresh

Rate Limiting:

Endpoint	Limit	Window
Login/Register	10 attempts	15 minutes
Password Change	5 attempts	1 hour
General API	100 requests	1 minute

System Architecture



DATA LAYER

PostgreSQL

Sequelize ORM

File Storage

Technology Stack & External Services

A comprehensive list of all technologies and external services powering Dispotree, with explanations for why each was chosen.

AI & Machine Learning Services

OpenAI GPT-4

LARGE LANGUAGE MODEL

Why: Powers the conversational AI agent for natural language property searches, deal analysis, and automated insights. GPT-4 provides the reasoning capability to understand complex queries like "find me deals in Texas with 20% equity under \$300K" and translate them into database operations.

LangChain

AI ORCHESTRATION FRAMEWORK

Why: Provides the agent framework that connects GPT-4 to our 20+ tools. Handles conversation memory, tool selection, and multi-step reasoning. Enables the AI to chain together multiple operations (search → score → analyze → recommend).

Pinecone

VECTOR DATABASE

Why: Stores document embeddings for semantic search in the knowledge base. When users ask questions, Pinecone finds the most relevant documents by meaning (not just keywords). Essential for RAG (Retrieval Augmented Generation).

TensorFlow.js

ML RUNTIME

Why: Runs neural network models directly in Node.js for deal quality prediction. No external API calls needed for ML inference, reducing latency. Trains custom models on fund feedback data to predict which deals will get accepted.

Database & Storage

PostgreSQL

RELATIONAL DATABASE

Why: Robust, ACID-compliant database for transactional data. Supports JSON columns for flexible schema (buy box criteria, workflow state). Excellent for complex queries needed for analytics and reporting.

Sequelize

ORM

Why: TypeScript-first ORM that provides type safety, migrations, and associations. Simplifies database operations while preventing SQL injection. Handles model relationships (User → BuyBox → Offers).

Document Processing

Tesseract.js

OCR ENGINE

Why: Extracts text from scanned documents and images. Enables searching through uploaded property

pdf-parse

PDF PROCESSING

Why: Extracts text from PDF contracts, disclosures, and reports. Enables contract analysis and searchable

photos, inspection reports, and handwritten notes. Runs locally without external API costs.

document archive. Fast, lightweight, no external dependencies.

Mammoth

WORD PROCESSING

Why: Converts Word documents (.docx) to HTML/text. Many contracts and templates arrive as Word files. Preserves formatting for contract template management.

xlsx

SPREADSHEET PROCESSING

Why: Parses Excel files for bulk property imports. Wholesalers often send property lists as Excel files. Handles complex spreadsheets with multiple sheets and formulas.

External Integrations

Firecrawl

WEB SCRAPING API

Why: Intelligent web scraping for auction sites and property listings. Handles JavaScript-rendered pages, pagination, and anti-bot measures. AI-powered extraction understands property data without manual CSS selectors.

Playwright

BROWSER AUTOMATION

Why: Automates browser actions for submitting offers to auction sites. Handles login flows, form submissions, and CAPTCHA scenarios. Used for browser-based integrations where APIs aren't available.

ImapFlow

EMAIL CLIENT

Why: Connects to email inboxes (Gmail, Outlook) to parse incoming deal emails. Monitors designated mailboxes for new property leads. Extracts attachments and parses email content automatically.

Google APIs

GOOGLE SERVICES

Why: Integration with Google Sheets for data import/export, Google Drive for document storage, and Gmail API for email processing. Many users already work in Google ecosystem.

Infrastructure & Security

Express.js

WEB FRAMEWORK

Why: Battle-tested Node.js framework. Minimal overhead, extensive middleware ecosystem. Easy to extend with custom middleware for auth, logging, and error handling.

Helmet

SECURITY MIDDLEWARE

Why: Adds security headers (CSP, X-Frame-Options, etc.) with one line of code. Protects against common web vulnerabilities. Industry standard for Express security hardening.

bcrypt

PASSWORD HASHING

Why: Industry-standard password hashing with automatic salting. 12 rounds provides strong security while maintaining acceptable performance. Protects user credentials even if database is compromised.

jsonwebtoken

JWT LIBRARY

Why: Stateless authentication tokens for API access. Short-lived access tokens (15 min) with longer refresh tokens (7 days). Enables secure, scalable authentication without session storage.

Validation & Data Quality

Joi

Zod

SCHEMA VALIDATION

Why: Declarative validation for API requests. Ensures data integrity before database operations. Provides clear error messages for invalid inputs.

TYPESCRIPT VALIDATION

Why: Type-safe validation with TypeScript inference. Used for complex nested objects and configuration validation. Generates TypeScript types from schemas.

AI Agent - Complete Tool Reference

The LangChain-powered AI agent provides a conversational interface to the entire system. Here's every tool available and what it can accomplish:

Property Management Tools

Tool	What It Does	Example Command
<code>searchProperties</code>	Find properties by state, city, price range, property type, equity percentage, or any combination	"Find SFRs in Texas under \$250K with 20%+ equity"
<code>getPropertyDetails</code>	Get full details on a specific property including all scores, documents, and history	"Show me details on property #1234"
<code>updateProperty</code>	Modify property fields (status, notes, asking price, etc.)	"Update property #1234 status to 'under contract'"
<code>addPropertyNote</code>	Add notes or comments to a property for team visibility	"Add note to #1234: 'Seller motivated, cash only'"

Buy Box & Scoring Tools

Tool	What It Does	Example Command
<code>listBuyBoxes</code>	Show all configured hedge fund buy boxes and their criteria	"What buy boxes do we have configured?"
<code>scoreProperty</code>	Score a property against all buy boxes or a specific fund	"How does #1234 score for Acme Capital?"
<code>findMatchingFunds</code>	Find which funds would be interested in a specific property	"Which funds would want this Texas property?"
<code>getBuyBoxAnalysis</code>	Analyze why a property matches or doesn't match a buy box	"Why did #1234 fail Fund X's buy box?"

Offer & Deal Tools

Tool	What It Does	Example Command
<code>createOffer</code>	Draft a new offer on a property	"Create an offer for \$180K on property #1234"

<code>listOffers</code>	View all offers on a property or by a user/fund	"Show all pending offers this week"
<code>updateOfferStatus</code>	Change offer status (accept, reject, counter)	"Mark offer #567 as accepted"
<code>compareOffers</code>	Compare multiple offers on the same property	"Compare the 3 offers on #1234"

Analytics & Reporting Tools

Tool	What It Does	Example Command
<code>getDashboardStats</code>	Get overview metrics (total deals, pending offers, conversion rates)	"What's our dashboard looking like today?"
<code>getConversionRates</code>	Calculate funnel metrics (view→like→offer→accept rates)	"What's our conversion rate this month?"
<code>getSourcePerformance</code>	Analyze which deal sources perform best	"Which source gives us the best deals?"
<code>getMarketTrends</code>	Analyze trends by market, property type, or time period	"How is the Texas market trending?"

Document & Contract Tools

Tool	What It Does	Example Command
<code>searchDocuments</code>	Semantic search through all uploaded documents	"Find documents mentioning 'roof repair'"
<code>analyzeContract</code>	AI analysis of contract terms, risks, and key dates	"Analyze the purchase agreement for #1234"
<code>listPropertyDocuments</code>	Show all documents attached to a property	"What documents do we have for #1234?"
<code>generateContract</code>	Create a contract from template with property data	"Generate a purchase agreement for offer #567"

Automation & Workflow Tools

Tool	What It Does	Example Command
<code>listAutomations</code>	Show all configured automations and their status	"What automations are active?"
<code>getWorkflowStatus</code>	Check status of running workflows	"What's the status of due diligence for #1234?"
<code>triggerWorkflow</code>	Manually start a workflow on a property	"Start due diligence workflow for #1234"

Knowledge Base Tools

Tool	What It Does	Example Command
<code>askKnowledgeBase</code>	Query the RAG knowledge base for company/process info	"What's our policy on dual closings?"
<code>searchKnowledge</code>	Find relevant documents in the knowledge base	"Find training docs about Texas title process"

Example Conversations

Multi-Step Analysis:

User: "Find me the best deal in Florida right now"
Agent: Searches properties → Scores against all buy boxes → Analyzes top matches → Returns recommendation with reasoning

Complex Query:

User: "Show me deals from CSV uploads this week that scored above 80 but haven't received any offers yet"
Agent: Filters by source → Filters by score → Filters by offer status → Returns matching properties with details

Week 2 Roadmap (Dec 20-26, 2025)

System Hardening - Making It Bulletproof

Focus Area	Planned Work
Input Validation	Comprehensive Joi/Zod schemas on all endpoints
Error Handling	Global error boundaries, graceful degradation
Database Security	Parameterized queries audit, connection pooling
API Protection	Request throttling, payload size limits, CORS hardening
Logging & Monitoring	Structured logging, error tracking, health dashboards
Testing Coverage	Edge cases, failure scenarios, load testing

ML Training & Prediction Models

ML Initiative	Description
---------------	-------------

Training Pipeline Enhancement	Automated retraining based on fund feedback accumulation
Model Accuracy Improvements	Feature engineering, hyperparameter tuning
Prediction Confidence Scores	Calibrated probability outputs for decision support
A/B Model Comparison	Side-by-side model performance evaluation
Real-time Predictions	Low-latency scoring for incoming deals
Fund-Specific Models	Custom models trained on individual fund preferences

Team Contributions

Team Member	Role	Key Contributions
Ed D.	Lead Developer & Architecture Design Lead	System architecture, plugin framework design, API structure, code review
Dejan P.	Developer	Core API development, database models, route implementations
Lennex Z.	Developer	Service integrations, testing infrastructure, documentation
Andy C.	ML Engineer	TensorFlow.js model development, deal quality scoring, training pipeline

Please let us know if you have any questions or would like a demo of any features.

Best regards,

Ed D.

Lead Developer & Architecture Design Lead
Dispotree Development Team

